

baseline

April 18, 2026

1 Baseline Modeling

1.1 Overview

This week, I will develop a baseline model for the classical TOPSIS method utilizing the PyDecision library. Subsequently, I will conduct several experiments, including stability analysis and an assessment of weight sensitivity.

1.2 Scenario

This round of testing will utilize data from Seattle Public Schools high schools. The objective is to determine which high school should be considered for closure based on several criteria, including student enrollment, budget allocation per student, ninth grade enrollment share, student retention rates, and grade distribution imbalances. Each high school will be evaluated as an alternative within the scenario, and school metrics will function as the assessment criteria.

1.3 Data Source

Information used in the baseline modeling was obtained from the Seattle Public High School Budget Summary 2024 and the Student Population Metrics for 2024–2025.

1.3.1 Data

- [Budget](#)
- [Population](#)

```
[8]: import pandas as pd
import numpy as np

from pyDecision.algorithm import topsis_method, ahp_method
from IPython.display import display

def load_population_data(file_path):
    try:
        return pd.read_csv(file_path)
    except Exception as e:
```

```

        print(f"Error loading school population data: {e}")
        return None

def load_budget_data(file_path):
    try:
        return pd.read_csv(file_path)
    except Exception as e:
        print(f"Error loading school budget data: {e}")
        return None

def build_features(population_data, budget_data):
    try:
        data = pd.merge(population_data, budget_data, on='High Schools',
        ↪how='inner')

        # Budget per student
        data['Budget per Student'] = data['Total Budget'] / data['Total_
        ↪Population']

        # Grade 9 share of total population
        data['Grade 9 Share'] = data['Grade 9'] / data['Total Population']

        # Student Retention
        data['Retention Rate'] = data['Grade 12'] / data['Grade 9']

        # Grade Imbalance
        data['Grade Imbalance'] = data[['Grade 9', 'Grade 10', 'Grade 11',
        ↪'Grade 12']].std(axis=1)

        return data
    except Exception as e:
        print(f"Error building features: {e}")
        return None

if __name__ == "__main__":
    population_data = load_population_data("../assets/basecase-data-population.
    ↪csv")
    budget_data = load_budget_data("../assets/basecase-data-budget.csv")

    if population_data is None or budget_data is None:
        raise ValueError("Failed to load required data files.")

    data = build_features(population_data, budget_data)

    # Define general testing criteria
    criteria = ['Total Population', 'Total Budget', 'Budget per Student',
    ↪'Grade 9 Share', 'Retention Rate', 'Grade Imbalance']

```

```

# What each criterion means for the decision (min for cost, max for benefit)
#
# - Total Population: Lower total population means less students affected
↳ by closure, which suggests stronger candidate for closure (cost)
# - Total Budget: Higher total budget means more resources to reallocate,
↳ which suggests stronger candidate for closure (benefit)
# - Budget per Student: Higher budget per student means more resources to
↳ reallocate, which suggests stronger candidate for closure (benefit)
# - Grade 9 Share: Lower grade 9 share means fewer incoming students, which
↳ suggests stronger candidate for closure (cost)
# - Retention Rate: Lower retention rate means more students leaving before
↳ graduation, which suggests stronger candidate for closure (cost)
# - Grade Imbalance: Higher grade imbalance means more uneven distribution
↳ of students across grades, which suggests stronger candidate for closure
↳ (benefit)
criteria_types = ['min', 'max', 'max', 'min', 'min', 'max']

print("\nMerged Data with Features:")
display(data[[
    'High Schools',
    'Total Population',
    'Total Budget',
    'Budget per Student',
    'Grade 9 Share',
    'Retention Rate',
    'Grade Imbalance'
]])

```

Merged Data with Features:

	High Schools	Total Population	Total Budget	Budget per Student	\
0	Ballard	1682	16205481	9634.649822	
1	Center School	150	3070819	20472.126667	
2	Cleveland STEM	778	9869983	12686.353470	
3	Franklin	1197	14174296	11841.517126	
4	Garfield	1436	14244718	9919.720056	
5	Ingraham	1348	15696088	11643.982196	
6	Lincoln	1749	15684047	8967.436821	
7	Nathan Hale	1005	12444002	12382.091542	
8	Rainier Beach	778	11047155	14199.428021	
9	Roosevelt	1536	14909173	9706.492839	
10	West Seattle	1450	13640324	9407.120000	

	Grade 9 Share	Retention Rate	Grade Imbalance
0	0.266350	0.926339	32.357379
1	0.293333	0.681818	6.244998

2	0.273779	0.694836	34.549481
3	0.239766	1.184669	30.159299
4	0.222841	1.181250	26.356530
5	0.237389	1.056250	13.904436
6	0.244711	0.915888	36.691280
7	0.216915	1.100917	27.293162
8	0.278920	0.843318	15.286159
9	0.252604	1.012887	13.490738
10	0.253103	0.950954	13.916417

1.4 TOPSIS Testing

Test	Weight Focus	Top Ranked Alternative	Score	Notes
Test 1	Equal weights	Cleveland STEM	0.621111	baseline
Test 2	Budget priority	Ballard	0.767190	favors high-budget schools
Test 3	Budget per student priority	Center School	0.752339	favors schools with high budget per student
Test 4	Population priority	Center School	0.798670	favors low-population closure candidates

1.4.1 Test 1

The initial assessment will serve as a general evaluation of the model's functionality. The weights will be distributed equally across each criterion during this process.

Assumption For this initial test there is no assumed outcome.

```
[9]: weights = [0.166, 0.166, 0.166, 0.166, 0.166, 0.166] # Equal weights for all
      ↪ criteria

scores = topsis_method(data[criteria], weights, criteria_types, False, False)
data['Topsis Score'] = scores
```

```
rankings = data[['High Schools', 'Topsis Score']].sort_values(by='Topsis_
↳Score', ascending=False)
print("\nTopsis Rankings:")
display(rankings)
```

Topsis Rankings:

	High Schools	Topsis Score
2	Cleveland STEM	0.621111
7	Nathan Hale	0.536578
3	Franklin	0.527412
1	Center School	0.506836
6	Lincoln	0.499900
8	Rainier Beach	0.490791
0	Ballard	0.489893
4	Garfield	0.451988
5	Ingraham	0.415067
9	Roosevelt	0.363142
10	West Seattle	0.356246

1.4.2 Test 2

The second test concentrates on the total budget criteria, examining how prioritizing the recovery of funds from the closed school would influence the rankings.

Asumption My assumption for this test is that the school with the highest budget will stand out.

```
[10]: weights = [0.1, 0.5, 0.1, 0.1, 0.1, 0.1] # Heavier weight on Total Budget

scores = topsis_method(data[criteria], weights, criteria_types, False, False)
data['Topsis Score'] = scores

rankings = data[['High Schools', 'Topsis Score']].sort_values(by='Topsis_
↳Score', ascending=False)
print("\nTopsis Rankings:")
display(rankings)
```

Topsis Rankings:

	High Schools	Topsis Score
0	Ballard	0.767190
6	Lincoln	0.754217
5	Ingraham	0.753770
3	Franklin	0.748841
4	Garfield	0.721888

9	Roosevelt	0.714184
10	West Seattle	0.674756
7	Nathan Hale	0.673355
8	Rainier Beach	0.583613
2	Cleveland STEM	0.539774
1	Center School	0.244438

1.4.3 Test 3

Asumption This test is to focus on budget per student and should rank the schools with the highest budget per student first.

```
[11]: weights = [0.1, 0.1, 0.5, 0.1, 0.1, 0.1] # Heavier weight on Budget per Student

scores = topsis_method(data[criteria], weights, criteria_types, False, False)
data['Topsis Score'] = scores

rankings = data[['High Schools', 'Topsis Score']].sort_values(by='Topsis_
↳Score', ascending=False)
print("\nTopsis Rankings:")
display(rankings)
```

Topsis Rankings:

	High Schools	Topsis Score
1	Center School	0.752339
8	Rainier Beach	0.462519
2	Cleveland STEM	0.400006
7	Nathan Hale	0.356420
3	Franklin	0.329398
5	Ingraham	0.284919
6	Lincoln	0.246479
0	Ballard	0.246217
4	Garfield	0.223598
9	Roosevelt	0.181581
10	West Seattle	0.167114

1.4.4 Test 4

Asumption This test is to focus on student population and should rank the schools with the highest population first.

```
[12]: weights = [0.5, 0.1, 0.1, 0.1, 0.1, 0.1] # Heavier weight on Total Budget

scores = topsis_method(data[criteria], weights, criteria_types, False, False)
```

```
data['Topsis Score'] = scores

rankings = data[['High Schools', 'Topsis Score']].sort_values(by='Topsis_
↪Score', ascending=False)
print("\nTopsis Rankings:")
display(rankings)
```

Topsis Rankings:

	High Schools	Topsis Score
1	Center School	0.798670
2	Cleveland STEM	0.609270
8	Rainier Beach	0.591325
7	Nathan Hale	0.475149
3	Franklin	0.375582
5	Ingraham	0.280409
4	Garfield	0.249788
10	West Seattle	0.219054
6	Lincoln	0.200035
0	Ballard	0.197315
9	Roosevelt	0.185960

1.4.5 Observations

The baseline TOPSIS results show that the ranking is sensitive to changes in criterion weights. Under equal weighting, Cleveland STEM ranked first, while the budget-focused configuration shifted Ballard to first place. When greater weight was assigned to the third and first criteria, Center School became the top-ranked alternative in both cases.

1.5 Ranking Stability

Spearman rank correlation was used to evaluate how stable the TOPSIS rankings remained under different weight configurations. The strongest agreement occurred between Test 3 and Test 4 ($=0.8909, p=0.000233$), indicating that these two weighting schemes produced very similar rankings. Test 1 also showed moderate to strong positive correlation with Test 3 ($=0.7636$) and Test 4 ($=0.6727$), suggesting that these three scenarios are relatively consistent with one another.

By contrast, Test 2 differed substantially from the other runs. Its correlations with Test 1, Test 3, and Test 4 were all negative, with the strongest disagreement observed between Test 2 and Test 4 ($= -0.7545, p=0.007282$). This indicates that the second weight configuration produced a ranking order that was largely inconsistent with the others.

These findings suggest that the baseline TOPSIS model has moderate stability overall, but is highly sensitive to certain weight changes. In particular, one weighting scenario caused a major shift in ranking behavior, showing that the final decision outcome depends strongly on the selected criterion priorities.

```

[13]: import pandas as pd
import numpy as np
from scipy.stats import spearmanr
from itertools import combinations

# -----
# Input your ranking results
# -----
test1 = pd.DataFrame({
    "High Schools": [
        "Cleveland STEM", "Nathan Hale", "Franklin", "Center School", "Lincoln",
        "Rainier Beach", "Ballard", "Garfield", "Ingraham", "Roosevelt", "West_
↪Seattle"
    ],
    "Topsis Score": [0.621111, 0.536578, 0.527412, 0.506836, 0.499900,
        0.490791, 0.489893, 0.451988, 0.415067, 0.363142, 0.356246]
})

test2 = pd.DataFrame({
    "High Schools": [
        "Ballard", "Lincoln", "Ingraham", "Franklin", "Garfield",
        "Roosevelt", "West Seattle", "Nathan Hale", "Rainier Beach",
        "Cleveland STEM", "Center School"
    ],
    "Topsis Score": [0.767190, 0.754217, 0.753770, 0.748841, 0.721888,
        0.714184, 0.674756, 0.673355, 0.583613, 0.539774, 0.244438]
})

test3 = pd.DataFrame({
    "High Schools": [
        "Center School", "Rainier Beach", "Cleveland STEM", "Nathan Hale",_
↪"Franklin",
        "Ingraham", "Lincoln", "Ballard", "Garfield", "Roosevelt", "West_
↪Seattle"
    ],
    "Topsis Score": [0.752339, 0.462519, 0.400006, 0.356420, 0.329398,
        0.284919, 0.246479, 0.246217, 0.223598, 0.181581, 0.167114]
})

test4 = pd.DataFrame({
    "High Schools": [
        "Center School", "Cleveland STEM", "Rainier Beach", "Nathan Hale",_
↪"Franklin",
        "Ingraham", "Garfield", "West Seattle", "Lincoln", "Ballard",_
↪"Roosevelt"
    ],
    "Topsis Score": [0.798670, 0.609270, 0.591325, 0.475149, 0.375582,

```

```

0.280409, 0.249788, 0.219054, 0.200035, 0.197315, 0.185960]
}))

tests = {
    "Test 1": test1,
    "Test 2": test2,
    "Test 3": test3,
    "Test 4": test4
}

# -----
# Convert each test into rank tables
# -----
rank_tables = {}

for name, df in tests.items():
    df = df.copy()
    df["Rank"] = range(1, len(df) + 1)
    rank_tables[name] = df

# -----
# Build combined rank + score table
# -----
schools = sorted(set().union(*[set(df["High Schools"]) for df in rank_tables.
    ↪values()])))

combined = pd.DataFrame({"High Schools": schools})

for name, df in rank_tables.items():
    combined = combined.merge(
        df[["High Schools", "Rank", "Topsis Score"]].rename(
            columns={"Rank": f"{name} Rank", "Topsis Score": f"{name} Score"}
        ),
        on="High Schools",
        how="left"
    )

print("Combined rank/score table:")
display(combined.sort_values("High Schools"))

```

Combined rank/score table:

	High Schools	Test 1 Rank	Test 1 Score	Test 2 Rank	Test 2 Score \
0	Ballard	7	0.489893	1	0.767190
1	Center School	4	0.506836	11	0.244438
2	Cleveland STEM	1	0.621111	10	0.539774
3	Franklin	3	0.527412	4	0.748841
4	Garfield	8	0.451988	5	0.721888

5	Ingraham	9	0.415067	3	0.753770
6	Lincoln	5	0.499900	2	0.754217
7	Nathan Hale	2	0.536578	8	0.673355
8	Rainier Beach	6	0.490791	9	0.583613
9	Roosevelt	10	0.363142	6	0.714184
10	West Seattle	11	0.356246	7	0.674756

	Test 3 Rank	Test 3 Score	Test 4 Rank	Test 4 Score
0	8	0.246217	10	0.197315
1	1	0.752339	1	0.798670
2	3	0.400006	2	0.609270
3	5	0.329398	5	0.375582
4	9	0.223598	7	0.249788
5	6	0.284919	6	0.280409
6	7	0.246479	9	0.200035
7	4	0.356420	4	0.475149
8	2	0.462519	3	0.591325
9	10	0.181581	11	0.185960
10	11	0.167114	8	0.219054

```
[14]: # -----
# Spearman rank correlation
# -----
test_names = list(rank_tables.keys())
spearman_results = []

for a, b in combinations(test_names, 2):
    temp = combined[["High Schools", f"{a} Rank", f"{b} Rank"]].dropna()
    rho, pval = spearmanr(temp[f"{a} Rank"], temp[f"{b} Rank"])
    spearman_results.append({
        "Comparison": f"{a} vs {b}",
        "Spearman rho": rho,
        "p-value": pval
    })

spearman_df = pd.DataFrame(spearman_results)
display(spearman_df.sort_values("Spearman rho", ascending=False))
```

	Comparison	Spearman rho	p-value
5	Test 3 vs Test 4	0.890909	0.000233
1	Test 1 vs Test 3	0.763636	0.006233
2	Test 1 vs Test 4	0.672727	0.023313
0	Test 1 vs Test 2	-0.354545	0.284693
3	Test 2 vs Test 3	-0.572727	0.065543
4	Test 2 vs Test 4	-0.754545	0.007282